Modern Service Mesh – a better way to secure communication between microservices

Stage 2: Proof of Concept

Jędrzej Bartoszewski Michał Domagała Kacper Maziarz Mikołaj Pawłoś

Selected Issues of Network Applications 2026/2027

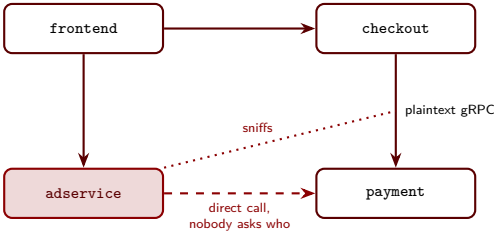
Faculty of Technical Physics, Information Technology and Applied Mathematics
Lodz University of Technology

November 2026

Inside the cluster, everyone trusts everyone

Kubernetes by default

- Online Boutique: 11 services talking gRPC and HTTP, all plaintext
- Any pod can call any other pod; nobody checks who's asking
- Firewalls guard the edge (North-South), and East-West traffic gets no checks at all



What an attacker gets from one weak pod

Attack	Why it works today
Eavesdropping	Traffic between pods is plaintext
Impersonation	Services have no cryptographic identity
Lateral movement	Nothing restricts who may call whom

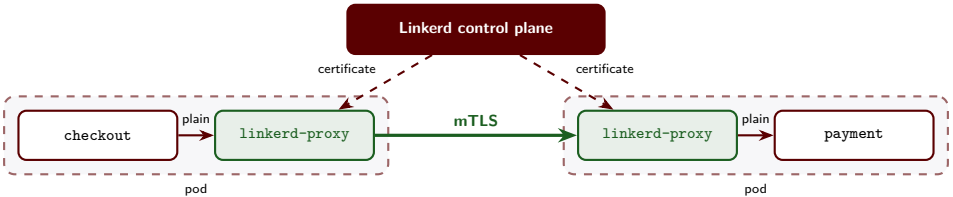
Three gaps, three different fixes

Encryption, identity and access control are separate problems. Stage 2 closes the first two; the third stays open until step 3.

Our idea: Zero Trust, enforced by the infrastructure

Service mesh

- A small proxy sits next to every service (a sidecar) and handles all its traffic
- Proxies encrypt with mutual TLS and prove identity with certificates
- Policies decide which identity may call which service
- **No change to application code**



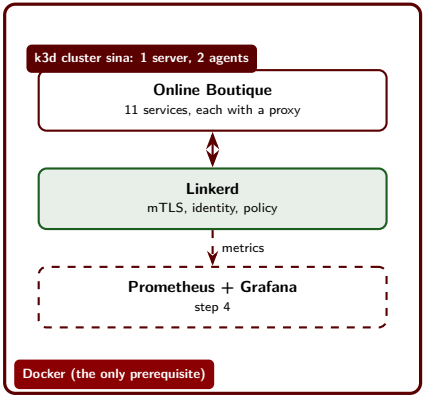
Why not the alternatives?

Option	Problem for us
TLS inside each app	Code changes in 5 languages, certificates managed by hand
Kubernetes NetworkPolicies	IP/port rules only: no encryption, no identity
Istio	Does everything, but heavier to run and to learn on a laptop cluster
Linkerd	Automatic mTLS, Rust micro-proxy, small enough for k3d

The stack

Tools

- **k3d:** Kubernetes in Docker containers, free and local
- **Online Boutique v0.10.7:** a realistic shop we attack instead of writing our own
- **Linkerd:** mTLS, identity, authorization policies
- **Prometheus + Grafana:** proxy metrics, dashboards for allowed vs. denied traffic



Where we are

Step	When	Status
1. k3d cluster + Online Boutique	weeks 3–4	**done**
2. Linkerd + mTLS proof of concept	weeks 5–6 (6.11)	**done**
3. Zero-Trust authorization policies	week 7	next
4. Prometheus + Grafana	week 8	planned
5. Attack simulation	week 9 (27.11)	planned

Step 1: one command, any laptop

Setup

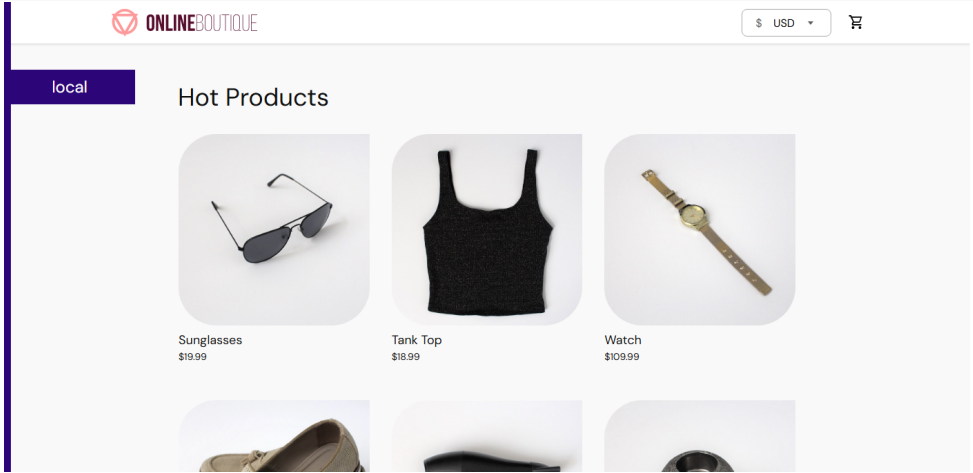
```
./setup.sh                      # Linux / macOS
powershell -File .\setup.ps1    # Windows
```

- Only Docker needed; k3d v5.9.0, kubectl v1.37.2 and Linkerd edge-26.11.1 land in `./.tools/`
- Safe to re-run; shop at `localhost:8080`

```
$ ./setup.sh
INFO[0027] Cluster 'sina' created successfully!
deployment.apps/adservice created
service/adservice created
serviceaccount/adservice created
deployment.apps/shippingservice created
# ... same for the other 10 services
```

```
NAME                                 READY   STATUS    RESTARTS   AGE
adservice-854578d668-xv5ls           1/1     Running   0          47s
checkoutservice-7c546c97fb-g4d2z     1/1     Running   0          47s
frontend-96b686b67-psvgp             1/1     Running   0          47s
paymentservice-9698c4f8d-fccxd       1/1     Running   0          46s
# ... 12 pods in total, all 1/1 Running: one container each, no proxy yet
```

Step 1: the shop, running locally



Online Boutique on our k3d cluster: 12 pods. Without the mesh, every call between them is plaintext

Step 2: is the traffic encrypted?

linkerd viz edges

```
$ linkerd viz edges deploy -n default | grep -v prometheus
```

SRC	DST	SRC_NS	DST_NS	SECURED
cartservice	redis-cart	default	default	√
checkoutservice	cartservice	default	default	√
checkoutservice	currencyservice	default	default	√
checkoutservice	emailservice	default	default	√
checkoutservice	paymentservice	default	default	√
checkoutservice	productcatalogservice	default	default	√
checkoutservice	shippingservice	default	default	√
frontend	adservice	default	default	√
frontend	cartservice	default	default	√
frontend	checkoutservice	default	default	√
frontend	currencyservice	default	default	√
frontend	productcatalogservice	default	default	√
frontend	recommendationservice	default	default	√
frontend	shippingservice	default	default	√
loadgenerator	frontend	default	default	√
recommendationservice	productcatalogservice	default	default	√

linkerd viz tap

```
$ linkerd viz tap deploy/checkoutservice --to deploy/paymentservice
req id=0:0 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true :...
rsp id=0:0 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true :...
end id=0:0 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true g...
req id=0:1 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true :...
rsp id=0:1 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true :...
end id=0:1 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true g...
req id=0:2 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true :...
rsp id=0:2 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true :...
end id=0:2 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true g...
```

Same request, before and after

Without mesh

```
$ tcpdump -A -s0 <payment pod traffic>

# outside the mesh (1/1), port 50051
20:02:34.002844 cni0  Out IP 10.42.2.13.38366 > 1...
E..m+.@.>..H
*.
*......
...:~?....@.......
.A....=...0.........+
.USD.........
.4432801561520454..... .
```

With mesh

```
$ tcpdump -A -s0 <payment pod traffic>

# inside the mesh (2/2), port 4143
20:05:00.123754 veth34f3e223 Out IP 10.42.2.13.33...
E.....@.@...
*.
*...R./....6.z....I.......
..../.F{....L..V`X`.L.Z...+1....Q.m..XeC4...#T.V....
$jQ...5U_...<.3..M..oV.b.8..P/6.
```

What surprised us

Encrypted ≠ protected

Any pod can still call `paymentservice`: Linkerd's default inbound policy is `all-unauthenticated`, so even unmeshed, plaintext clients get in.

Smaller lessons

- Traefik, on by default in k3s, clashes with the Gateway API CRDs Linkerd needs, so we disabled it
- Showing encryption took more work than turning it on: distroless images, `runAsNonRoot`, so we captured from the node
- A full system drive hung Docker Desktop, and the setup script with it

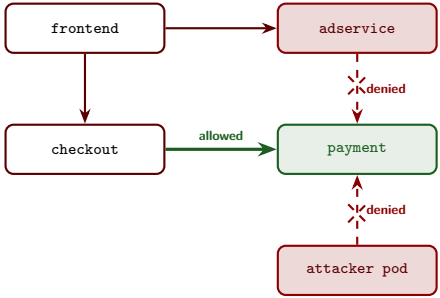
Next: close the door (steps 3 and 5)

Authorization policies

- Default-deny for the namespace, then allow only the calls the app really makes
- Only the `checkoutservice` identity may call `paymentservice`

Attack simulation

Same attack twice, from an outside pod and from a hijacked `adservice`: it goes through now, and should get `PermissionDenied` after.



The other 12 allowed edges of the call graph are left out.

Next: make it visible (step 4)

Observability

- Prometheus scrapes every proxy: request counts, latency, TLS status, denials
- Grafana dashboard showing allowed vs. denied requests per service
- The attack from step 5 should show up on it live

Plan

Start with the Prometheus bundled with `linkerd-viz` and add Grafana; move to a separate Prometheus if it limits us

Thank you

Questions?